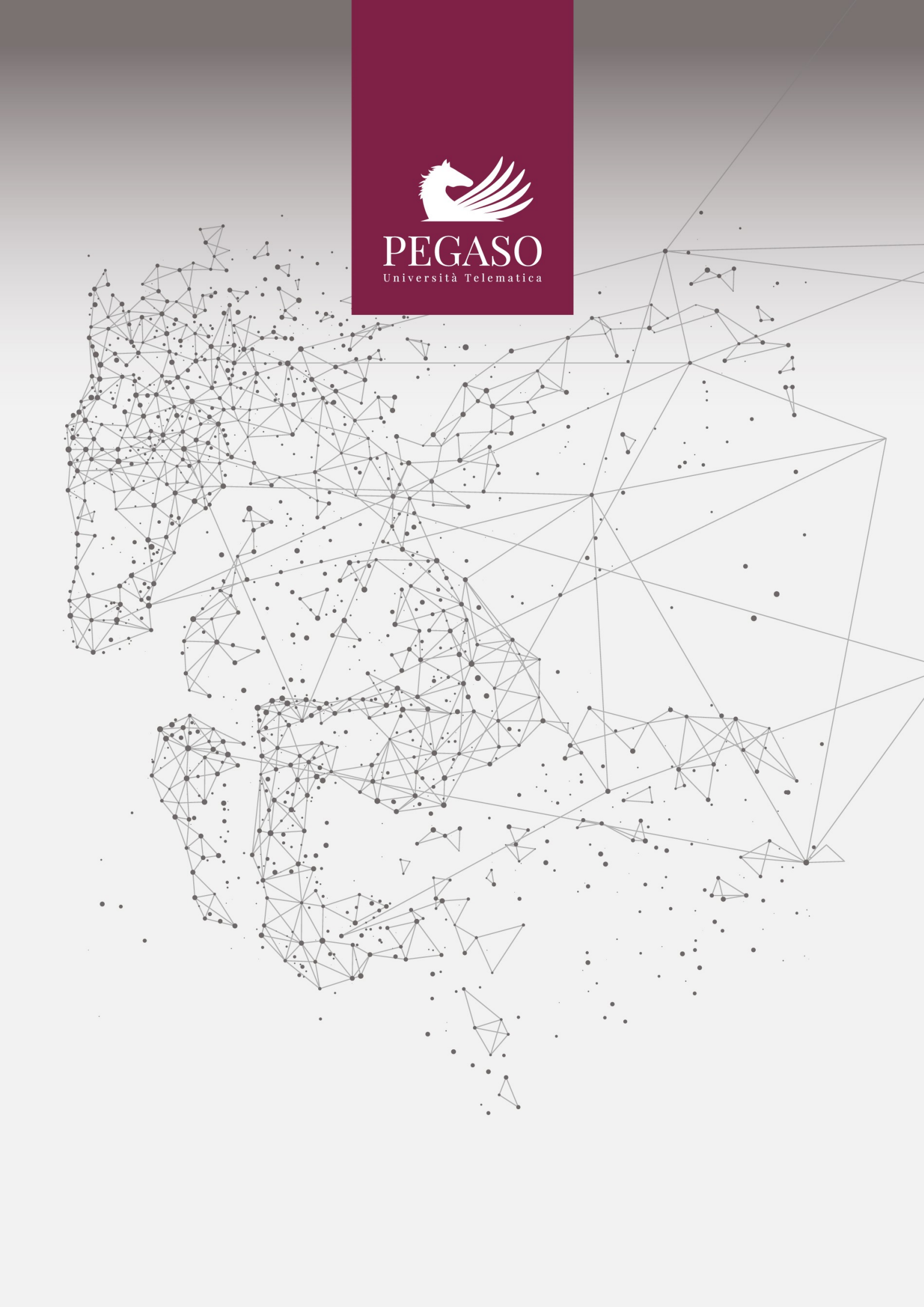

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. IL PROBLEMA DELL'ORDINAMENTO 3

2. ALGORITMI DI ORDINAMENTO 6

3. THE SOUND OF SORTING.....8

BIBLIOGRAFIA E SITOGRAFIA 10

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Il problema dell'ordinamento

Il problema dell'ordinamento può essere definito "formalmente" nella seguente maniera: data una sequenza di n numeri $\langle a_1, a_2, \ldots, a_n \rangle$ un ordinamento è una permutazione (un ri-arrangiamento) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ della sequenza di input tale che $1 \leq a'_1 \leq a'_2 \leq \cdots \leq a'_n$

Più semplicemente possiamo dire che l'ordinamento di una sequenza di informazioni consiste nel disporre le stesse informazioni in modo da rispettare una qualche relazione d'ordine di tipo lineare (ad esempio una relazione d'ordine "minore o uguale" dispone le informazioni in modo "non decrescente").

Solitamente, ogni numero da ordinare appartiene ad una raccolta di dati detta record ed ogni record contiene una chiave, che è il valore da ordinare; la parte restante del record è composta da dati satelliti, che di solito si muovono con la chiave. Da notare che la chiave può essere costituita anche da un numero finito di attributi di ciascuna informazione: in tal caso una ricerca di un elemento all'interno della raccolta, può trarre vantaggio dall'ordinamento se la ricerca stessa viene effettuata utilizzando le stesse chiavi sulle quali è stata applicata la relazione d'ordine. Generalmente le chiavi di ordinamento vengono scelte in modo da permettere di distinguere tutti gli elementi dell'insieme da ordinare; ad esempio, se si dovesse ordinare un archivio anagrafico, la scelta delle sole chiavi "cognome" e "nome" non permetterebbe di distinguere (sia nella fase di ordinamento sia in quella di ricerca), persone con lo stesso cognome e nome.

Quando un algoritmo di ordinamento permuta le chiavi, deve permutare anche i dati satelliti: se ogni record include una grande quantità di dati satellite, spesso permutiamo un array di puntatori ai record, anziché i record stessi, per minimizzare gli spostamenti dei dati.

In un certo senso, sono questi dettagli implementativi che distinguono un algoritmo da un programma vero e proprio: ordinare singoli numeri o grandi record che contengono numeri non è rivelante per il metodo in base al quale una procedura di ordinamento determina l'ordine degli elementi. Pertanto, quando analizziamo un problema di ordinamento, tipicamente supponiamo che l'input sia composto soltanto da numeri. La conversione di un algoritmo che ordina numeri in un programma che ordina record è concettualmente semplice, sebbene in determinati casi alcuni dettagli tecnici potrebbero complicare la programmazione effettiva.

Il motivo per cui l'ordinamento genera un grande interesse è dovuto a diversi fattori:

- A volte l'esigenza di ordinare le informazioni è inerente a un'applicazione (ordinare gli studenti di una scuola in base alle medie).

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Gli algoritmi spesso usano l'ordinamento come una subroutine chiave: ad esempio ordinare i livelli su un'applicazione di grafica o le tracce per un'applicazione audio.
- Esistono vari tipi di algoritmi di ordinamento, che impiegano un'ampia gamma di tecniche.
- Molti problemi di progettazione possono essere affrontati usando tecniche di ordinamento anziché andando ad "aggiustare" il codice.

C'è da precisare che alcuni algoritmi richiedono solide basi matematiche (anche avanzate) che e concetti di analisi probabilistica, algoritmi randomizzati ed altre tecniche sofisticate di risoluzione.

Oltre che per il loro principio di funzionamento e per la loro efficienza, gli algoritmi di ordinamento possono essere confrontati in base ai seguenti criteri:

- **Stabilità**: un algoritmo di ordinamento è stabile se non altera l'ordine relativo di elementi dell'array aventi la stessa chiave. Algoritmi di questo tipo evitano interferenze con ordinamenti pregressi dello stesso array basati su chiavi secondarie. Se ad esempio si ordina per anno di corso una lista di studenti già ordinata alfabeticamente, un metodo stabile produce una lista in cui gli alunni dello stesso anno sono ancora in ordine alfabetico mentre un ordinamento instabile probabilmente produrrà una lista senza più alcuna traccia del precedente ordinamento.
- **Sul posto (in place)**: un algoritmo di ordinamento opera in place se la dimensione delle strutture ausiliarie di cui necessita è indipendente dal numero di elementi dell'array da ordinare. In altre parole: un algoritmo in place non crea una copia dell'input per raggiungere l'obiettivo (l'ordinamento), pertanto un algoritmo in place risparmia memoria rispetto ad un algoritmo non in place. Si intuisce quanto sui grandi numeri la proprietà "in place" sia rilevante.

  In termini di classificazione, gli algoritmi di ordinamento possono essere distinti in due classi:
- **Algoritmi di Ordinamento Interno**: fanno uso di strutture dati allocate in memoria centrale dell'elaboratore, per tale motivo hanno il principale limite di non permettere l'ordinamento di grosse sequenze di informazioni.
- **Algoritmi di Ordinamento Esterno**: tipicamente fanno uso di strutture file per la memorizzazione e le operazioni di ordinamento della sequenza di informazioni da ordinare.

È inoltre possibile classificare in base alla complessità del tempo di calcolo. La complessità di calcolo si riferisce soprattutto al numero di operazioni necessarie all'ordinamento (principalmente operazioni di confronto e scambio), in funzione del numero di elementi da ordinare:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Algoritmi Semplici di Ordinamento**: algoritmi che presentano una complessità proporzionale a n^2, essendo n è il numero di informazioni da ordinare; essi sono generalmente caratterizzati da poche e semplici istruzioni.
- **Algoritmi Evoluti di Ordinamento**: algoritmi che offrono una complessità computazionale proporzionale ad $n \log_2 n$ (che è sempre inferiore a n^2). Tali algoritmi sono molto più complessi e fanno molto spesso uso di ricorsione. La convenienza del loro utilizzo si ha unicamente quando il numero n di informazioni da ordinare è molto elevato.

Da precisare che è possibile dimostrare che il problema dell'ordinamento non può essere risolto con un algoritmo di complessità asintotica inferiore a quella pseudo-lineare: per ogni algoritmo che ordina un array di n elementi, il tempo d'esecuzione soddisfa $T(n) = \Omega(n \cdot \log_2 n)$.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Algoritmi di ordinamento

È possibile individuare in letteratura una lista molto lunga di algoritmi di ordinamento; quelli di particolare nel nostro contesto di lavoro sono i seguenti:

- Insertsort.
- Selectsort.
- Bubblesort.
- Mergesort.
- Quicksort.
- Heapsort.

Riportiamo schematicamente le caratteristiche di tali algoritmi in termini di complessità e criteri di confronto:

Nome	***Migliore***	***Medio***	***Peggiore***	***Memoria***	***Stabile***	***In place***
Bubble sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Sì	Sì
Heap sort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$\Theta(1)$	No	Sì
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Sì	Sì
Merge sort	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$O(n)$	Sì	No
Quick sort	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$O(n^2)$	$O(n)$	No	Sì
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	No	Sì

Ricordiamo brevemente il significato delle notazioni asintotiche:

- **Notazione asintotica O** (notazione O grande): limite superiore asintotico.
- **Notazione asintotica Ω** (notazione Omega): limite inferiore asintotico.
- **Notazione asintotica θ** (notazione Theta): limite asintotico stretto (se una funzione è $\theta(g(n))$ allora è anche $O(g(n))$ e $\Omega(g(n))$)

Schematicamente possiamo dire che:

- **Selection sort**: semplice, intuitivo ma poco efficiente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Bubble sort**: semplice, abbastanza intuitivo ma appena poco più efficiente di Selection sort.
- **Insertion sort**: intuitivo ed abbastanza efficiente.
- **Quick sort**: non intuitivo ma alquanto efficiente.
- **Merge sort**: non intuitivo ma molto efficiente.
- **Heap sort**: non intuitivo ma molto efficiente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. The sound of sorting

Timo Bingmann, attualmente Senior Member Of Technical Staff di eBay, nel 2013, durante il suo dottorato nel Karlsruhe Institute of Technology, ha pubblicato sul suo canale YouTube un video dal titolo **15 Sorting Algorithms in 6 Minutes**[1]. Nel video vengono visualizzati e sonorizzati 15 algoritmi di ordinamento.

Immagini e suoni ci danno la possibilità di "percepire" esattamente come si comportano alcuni degli algoritmi che abbiano enunciato in precedenza e che saranno oggetto di analisi.

Il video si apre con il Selection Sort, l'algoritmo di ordinamento più intuitivo e che quello che qualunque operatore umano sfrutterebbe. Proviamo ad immaginare di essere di fronte a un tavolo da biliardo con tutte le palle sul tavolo, disposte a caso. Se volessimo ordinarle, una tecnica potrebbe essere quella di individuare quella con il numero più alto (selection) e metterla "in fondo" a una ipotetica fila, proseguendo nella stessa modalità con le palle rimanenti. Questo è quello che fa l'algoritmo nel video: seleziona l'asticella più bassa, la mette all'inizio, e poi itera il procedimento sugli elementi restanti. È per questo motivo che si vede un triangolo prendere forma dalla punta (da sinistra verso destra).

Intorno al minuto 4:00 c'è il Bubble Sort. Tale algoritmo confronta un elemento con il successivo: se il primo è maggiore del secondo, li scambia, e questo lo fa per tutti gli elementi. Questa operazione ha effetto di portare l'elemento massimo in fondo, una volta visitati tutti gli elementi e poi ripete l'operazione sugli elementi privati dell'elemento in fondo, e così via, su insiemi sempre più piccoli.

Il triangolo delle asticelle ordinate prende forma da destra verso sinistra, poiché gli elementi massimi vengono via via accumulati in fondo.

Intorno ai 40 secondi c'è il Quick Sort: questo è uno degli algoritmi di ordinamento più efficienti, forse il più efficiente nel caso medio ed è anche quello che è implementato nella stragrande maggioranza delle API (Application Programming Interface) informatiche.

L'algoritmo sceglie un elemento chiamato PIVOT all'interno dell'insieme che viene organizzato in modo da ottenere due partizioni: la prima conterrà tutti i numeri minori o uguali del pivot, la seconda tutti i numeri maggiori o uguali ("numero minore" diventa "asticella più bassa" nel video). Attenzione: le due partizioni sono ancora disordinate al loro interno; l'unico che è al suo posto è proprio il pivot.

A questo punto entra in gioco la ricorsione, ovvero, la stessa tecnica la si applica per le due partizioni. Quando non rimangono più partizioni, l'algoritmo termina.

[1] The Sound of Sorting - "Audibilization" and Visualization of Sorting Algorithms - panthema.net

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel video è evidente la fase di partizionamento, che lavora via via su partizioni sempre più piccole: il triangolo viene costruito "a blocchi" (le partizioni) di granularità sempre più fine.

Il Merge Sort che compare intorno al minuto ha ancora una soluzione ricorsiva. Divide l'insieme a metà, poi le due metà le divide a metà, e così via... Fino ad arrivare a tante "metà" di solo due elementi ciascuna. Le metà vengono ordinate scambiando gli elementi se il primo è maggiore del secondo, altrimenti lasciandoli nel loro ordine (sort); a questo punto le metà vengono fuse a due a due (merge) avendo cura di mantenere l'ordinamento. Si ripete ricorsivamente il procedimento.

Seguendo questo pattern, nel video, il triangolo grande prende forma tramite tanti triangolini (sottoinsiemi ordinati) che vengono via via fusi (mediante il merge).

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley
- Sedgewick: Algoritmi in C, Pearson, 2015
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione
- The Sound of Sorting - "Audibilization" and Visualization of Sorting Algorithms - panthema.net

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).